

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 2

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan struktur data Array List di Java. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman struktur data Array List di java.

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan struktur data Array List di Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Praktikum	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	2
 BAB II PEMBAHASAN.....	 3
2.1 Arraylist_2511532010	3
2.2 Mahasiswa_2511532010 dan MahasiswaDriver_2511532010	5
2.3 DaftarKata_2511532010 dan DaftarKataDriver_2511532010.....	11
 BAB III KESIMPULAN.....	 16
3.1 Ringkasan Hasil Praktikum.....	16
DAFTAR PUSTAKA.....	17
TUGAS	18

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Dalam pengembangan aplikasi berbasis Java, efisiensi pengelolaan kumpulan data merupakan aspek yang sangat krusial. Penggunaan struktur Array konvensional sering kali menemui hambatan karena sifatnya yang statis, di mana kapasitas penyimpanan harus ditentukan di awal dan tidak dapat diubah selama eksekusi program. Kondisi ini menjadi kendala signifikan saat pengembang menghadapi situasi dengan volume data yang bersifat dinamis atau sulit diprediksi secara pasti.

Sebagai solusinya, Java menghadirkan ArrayList dalam framework Collection sebagai alternatif yang lebih fleksibel. ArrayList merupakan implementasi array yang mampu menyesuaikan ukurannya secara otomatis (resizable-array) sesuai dengan jumlah data yang diinputkan. Dengan mempelajari mekanisme operasionalnya—seperti proses penambahan, penghapusan, dan pengaksesan elemen—praktikan dapat merancang aplikasi yang lebih adaptif dan optimal dalam manajemen memori tanpa perlu mengkhawatirkan batasan alokasi ruang yang bersifat tetap.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk membekali praktikan dengan pemahaman komprehensif mengenai inisialisasi dan pengoperasian struktur data ArrayList pada bahasa pemrograman Java. Fokus utamanya adalah penguasaan implementasi metode standar seperti add(), get(), set(), dan remove(). Selain itu, praktikan diarahkan untuk memahami mekanisme tipe data generik yang berfungsi menjamin keamanan tipe (type safety) dalam mengelola koleksi data.

Di samping aspek teknis, praktikum ini dirancang untuk mengasah logika berpikir dalam menangani iterasi serta manipulasi data yang bersifat kompleks. Melalui

serangkaian latihan, praktikan ditargetkan memiliki kemampuan analitis untuk menentukan urgensi penggunaan ArrayList dibandingkan struktur data lain. Hal ini juga mencakup strategi optimasi performa program saat mengelola volume data besar, khususnya melalui penerapan teknik sorting dan searching yang efisien.

1.3 Manfaat Praktikum

Secara teknis, praktikum ini bermanfaat dalam mengasah kemahiran pemrograman untuk mengelola struktur data dinamis yang relevan dengan kebutuhan industri. Melalui penguasaan ini, praktikan mampu membangun aplikasi yang lebih adaptif dan stabil, bahkan saat menghadapi fluktuasi jumlah data yang ekstrem. Implementasi yang tepat juga berperan penting dalam meminimalisir kesalahan fatal pada sistem, seperti risiko error `ArrayIndexOutOfBoundsException` yang sering terjadi pada array statis.

Secara konseptual, pemahaman terhadap ArrayList berfungsi sebagai fondasi krusial untuk mempelajari struktur data yang lebih progresif, seperti `LinkedList`, `Stacks`, hingga `Queues`. Dengan menguasai prinsip dasar manipulasi objek dalam sebuah koleksi, praktikan akan lebih siap dalam mengadopsi berbagai library eksternal maupun framework modern Java. Hal ini dikarenakan ekosistem pengembangan Java saat ini sangat bergantung pada prinsip-prinsip utama yang terdapat dalam `Java Collection Framework`.

BAB II

PEMBAHASAN

2.1 ArrayList_2511532010

```
1 package pekan2_2511532010;
2 import java.util.ArrayList;
3 public class ArrayList_2511532010 {
4
5     public static void main(String[] args) {
6         int n=5;
7         //Declaring the ArrayList with initial size n
8         ArrayList<Integer> arrli = new ArrayList<Integer>(n);
9         // Appending new elements at the end of the list
10        for (int i=1; i<=n; i++)
11            arrli.add(i);
12        // Printing elements
13        System.out.println(arrli);
14        // Remove element at index 3
15        arrli.remove(3);
16        // Displaying the ArrayList
17        // after deletion
18        System.out.println(arrli);
19        // printing elements one by one
20        for (int i=0; i< arrli.size(); i++)
21            System.out.print(arrli.get(i) + " ");
22
23
24
25    }
26
27 }
```

1. Inisialisasi Dynamic Array

Pada baris `ArrayList<Integer> arrli = new ArrayList<Integer>(n)`, program membuat sebuah objek daftar dinamis.

- Generics (<Integer>): ArrayList menggunakan konsep Generics, yang berarti ia hanya bisa menyimpan tipe data objek. Karena itu, digunakan Integer (Wrapper Class) dan bukan tipe data primitif int.

- Kapasitas Awal: Angka `n` di dalam kurung menentukan kapasitas awal (10 slot jika tidak ditentukan). Namun, jika data yang dimasukkan melebihi kapasitas ini, Java akan otomatis memperbesar ukurannya di latar belakang.

2. Penambahan Elemen (add)

Program menggunakan metode `arrli.add(i)` di dalam perulangan.

- Metode ini bekerja dengan menambahkan data ke urutan paling akhir dari daftar.
- Proses ini jauh lebih sederhana dibandingkan array biasa karena kita tidak perlu melacak indeks mana yang masih kosong; `ArrayList` mengelola indeksnya sendiri secara internal.

3. Logika Penghapusan (remove)

Baris `arrli.remove(3)` memerintahkan program untuk menghapus elemen pada indeks ke-3.

- Penting: Indeks dimulai dari 0. Jika list berisi `[1, 2, 3, 4, 5]`, maka elemen pada indeks ke-3 adalah angka 4.
- Automatic Shifting: Keunggulan utama `ArrayList` terlihat di sini. Saat elemen dihapus, elemen-elemen di belakangnya (dalam hal ini angka 5) akan bergeser ke kiri secara otomatis untuk mengisi celah yang kosong. kita tidak perlu melakukan perulangan manual seperti pada program `Driver_2511532010` sebelumnya.

4. Pengukuran Ukuran (size)

Pada perulangan terakhir, digunakan `arrli.size()`.

- Berbeda dengan array biasa yang menggunakan properti `.length` (mengukur total kapasitas), `size()` pada `ArrayList` mengembalikan jumlah elemen yang benar-benar ada di dalam daftar.
- Setelah penghapusan, `size()` yang awalnya 5 akan otomatis berubah menjadi

5. Akses Elemen (get)

Metode `arrli.get(i)` digunakan untuk mengambil nilai pada indeks tertentu.

- Pada array standar, kita menggunakan kurung siku `arr[i]`.
- Pada `ArrayList`, akses data harus dilakukan melalui metode `get()`. Ini adalah bagian dari penerapan metode akses yang aman dalam pemrograman berorientasi objek.

HASIL :

```
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
1 2 3 5
```

2.2 Mahasiswa_2511532010 dan MahasiswaDriver_2511532010

```
1 package pekan2_2511532010;
2
3 public class Mahasiswa_2511532010 {
4
5     String nama;
6     String nim;
7     String prodi;
8     Mahasiswa_2511532010(String nim, String nama, String prodi) {
9         this.nim = nim;
10        this.nama = nama;
11        this.prodi = prodi;
12    }
13
14    @Override
15
16    public String toString() {
17        return "Nim: " + nim + ", Nama: " + nama + ", Prodi: " + prodi;
18    }
19 }
```



```

1 package pekan2_2511532010;
2 import java.util.Scanner;
3 import java.util.ArrayList;
4
5 public class MahasiswaDriver_2511532010 {
6     public static void tampilkanMenu() {
7         System.out.println("\nMenu:");
8         System.out.println("1. Tambah Mahasiswa");
9         System.out.println("2. Tampilkan Semua Mahasiswa");
10        System.out.println("3. Hapus Mahasiswa Berdasarkan NIM");
11        System.out.println("4. Cari Mahasiswa Berdasarkan NIM");
12        System.out.println("5. Keluar");
13    }
14
15    public static void tambahMahasiswa_2511532010(ArrayList<Mahasiswa_2511532010> list, Scanner sc) {
16        System.out.print("Masukkan NIM: ");
17        String nim = sc.nextLine();
18
19        System.out.print("Masukkan Nama: ");
20        String nama = sc.nextLine();
21
22        System.out.print("Masukkan Prodi: ");
23        String prodi = sc.nextLine();
24
25        list.add(new Mahasiswa_2511532010(nim, nama, prodi));
26        System.out.println("Mahasiswa berhasil ditambahkan.");
27    }
28
29    public static void tampilkanSemuaMahasiswa_2511532010(ArrayList<Mahasiswa_2511532010> list) {
30        if (list.isEmpty()) {
31            System.out.println("Daftar mahasiswa kosong.");
32        } else {
33            System.out.println("\n--- Data Mahasiswa ---");
34            for (Mahasiswa_2511532010 mhs : list) {
35                System.out.println(mhs);
36            }
37        }
38    }
39
40    public static void hapusMahasiswa_2511532010(ArrayList<Mahasiswa_2511532010> list, Scanner sc) {
41        System.out.print("Masukkan NIM yang akan dihapus: ");
42        String nimHapus = sc.nextLine();
43        boolean removed = list.removeIf(mhs -> mhs.nim.equals(nimHapus));
44
45        if (removed) {
46            System.out.println("Data dengan NIM " + nimHapus + " berhasil dihapus.");
47        } else {
48            System.out.println("NIM tidak ditemukan.");
49        }
50    }
51
52    public static void cariMahasiswa_2511532010(ArrayList<Mahasiswa_2511532010> list, Scanner sc) {
53        System.out.print("Masukkan NIM yang dicari: ");
54        String nimCari = sc.nextLine();
55        boolean ditemukan = false;
56
57        for (Mahasiswa_2511532010 mhs : list) {
58            if (mhs.nim.equals(nimCari)) {
59                System.out.println("Hasil Pencarian: " + mhs);
60                ditemukan = true;
61                break;
62            }
63        }
64        // Perbaikan logika: jika TIDAK ditemukan (!ditemukan)
65        if (!ditemukan) {
66            System.out.println("Mahasiswa dengan NIM " + nimCari + " tidak ditemukan.");
67        }
68    }
69
70    public static void main(String[] args) {
71        ArrayList<Mahasiswa_2511532010> mahasiswaList = new ArrayList<>();
72        Scanner scanner = new Scanner(System.in);
73        int choice = 0;
74
75        do {
76            tampilkanMenu();
77            System.out.print("Pilih menu : ");
78
79            // Validasi agar tidak error jika input bukan angka
80            if (scanner.hasNextInt()) {
81                choice = scanner.nextInt();
82                scanner.nextLine(); // PENTING: Membersihkan buffer newline setelah nextInt()
83            } else {
84                System.out.println("Input harus berupa angka!");
85                scanner.nextLine(); // Membersihkan input yang salah
86                continue;
87            }
88
89            switch (choice) {
90                case 1:
91                    tambahMahasiswa_2511532010(mahasiswaList, scanner);
92                    break;
93                case 2:
94                    tampilkanSemuaMahasiswa_2511532010(mahasiswaList);
95                    break;
96                case 3:
97                    hapusMahasiswa_2511532010(mahasiswaList, scanner);
98                    break;
99                case 4:
100                   cariMahasiswa_2511532010(mahasiswaList, scanner);
101                   break;
102                case 5:
103                   System.out.println("Program selesai. Sampai jumpa!");
104                   break;
105                default:
106                   System.out.println("Pilihan tidak valid.");
107            }
108        } while (choice != 5);
109
110        scanner.close();
111    }
112 }

```

1. Struktur Objek Mahasiswa

Kelas Mahasiswa_2511532010 bertindak sebagai model data. Di sini kita mendefinisikan tiga variabel String (nama, nim, prodi). Pemilihan tipe data String untuk NIM sangat tepat karena NIM seringkali diawali dengan angka nol atau mengandung karakter yang tidak memerlukan operasi matematika.

2. Constructor Berparameter

Terdapat konstruktor Mahasiswa_2511532010(String nim, String nama, String prodi). Fungsi ini memungkinkan pembuatan objek mahasiswa yang lengkap dalam satu baris. Penggunaan kata kunci this (seperti this.nim = nim) berfungsi untuk membedakan antara variabel milik kelas dengan parameter yang dikirimkan melalui input.

3. Overriding Method toString

kita melakukan *override* pada metode toString(). Tanpa metode ini, saat kita mencetak objek, Java hanya akan menampilkan alamat memori (hashcode). Dengan adanya *override*, objek mahasiswa akan dicetak dalam format teks yang rapi sehingga mudah dibaca saat menu "Tampilkan Semua" dijalankan.

4. Penggunaan ArrayList of Objects

Pada kelas driver, kita mendeklarasikan ArrayList<Mahasiswa_2511532010>. Ini adalah struktur data dinamis yang jauh lebih fleksibel dibandingkan Array biasa. Kita tidak perlu menentukan kapasitas maksimal di awal, list ini akan membesar otomatis di memori komputer sesuai dengan jumlah mahasiswa yang kita inputkan.

5. Manajemen Input dan Buffer Flush

Dalam metode main, kita menggunakan scanner.nextLine() setelah scanner.nextInt(). Ini adalah langkah krusial untuk "membersihkan" sisa karakter

newline (\n) di memori. Jika ini tidak dilakukan, input String pada menu berikutnya akan otomatis terlewati (bug yang sering terjadi pada pemula Java). Validasi Input Angka (Input Safety)

Terdapat pengecekan if (`scanner.hasNextInt()`). Ini berfungsi agar program tidak langsung *error* atau *force close* jika pengguna tidak sengaja memasukkan karakter huruf pada menu pilihan. Jika input bukan angka, program akan memberikan peringatan dan meminta input ulang.

6. Modularitas dengan Static Methods

Program kita sangat rapi karena menggunakan metode statis seperti `tambahMahasiswa`, `hapusMahasiswa`, dan `cariMahasiswa`. Pemisahan ini membuat kode di dalam `main` menjadi sangat pendek dan mudah dibaca, karena logika utama dipindahkan ke fungsi-fungsi spesifik di luar `main`.

7. Penghapusan Modern dengan Lambda

Pada fungsi `hapusMahasiswa`, kita menggunakan `list.removeIf(mhs -> mhs.nim.equals(nimHapus))`. Ini adalah fitur Java 8+ yang sangat efisien. Program secara cerdas akan memindai seluruh isi list dan langsung menghapus objek yang NIM-nya cocok, tanpa kita perlu menulis perulangan `for` secara manual.

8. Logika Pencarian (Sequential Search)

Metode `cariMahasiswa` menggunakan perulangan *for-each*. Di sini terdapat logika pengecekan NIM menggunakan `.equals()`. Karena NIM adalah String, perbandingan tidak boleh menggunakan `==`. Jika NIM ditemukan, sistem langsung menampilkan data mahasiswa tersebut dan berhenti mencari menggunakan perintah `break`.

9. Perbaikan Logika Flagging

Pada bagian pencarian, kita menggunakan variabel boolean ditemukan. Ini adalah teknik *flagging* yang bagus. Jika setelah seluruh list diperiksa nilai ditemukan masih false (menggunakan logika !ditemukan), barulah program mencetak pesan bahwa data tidak ada. Ini mencegah munculnya pesan "Tidak Ditemukan" berkali-kali di dalam perulangan.

HASIL :

```

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu : 1
Masukkan NIM: 2511533663
Masukkan Nama: natalie
Masukkan Prodi: sejarah
Mahasiswa berhasil ditambahkan.

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu : 1
Masukkan NIM: 27177172
Masukkan Nama: natasha
Masukkan Prodi: ilkom
Mahasiswa berhasil ditambahkan.

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu : 2

--- Data Mahasiswa ---
Nim: 2511533663, Nama:natalie, Prodi: sejarah
Nim: 27177172, Nama:natasha, Prodi: ilkom

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu : 3
Masukkan NIM yang akan dihapus: 27177172
\2Data dengan NIM 27177172 berhasil dihapus.

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu :
Input harus berupa angka!

Menu:
1. Tambah Mahasiswa
2. Tampilkan Semua Mahasiswa
3. Hapus Mahasiswa Berdasarkan NIM
4. Cari Mahasiswa Berdasarkan NIM
5. Keluar
Pilih menu : 2

--- Data Mahasiswa ---
Nim: 2511533663, Nama:natalie, Prodi: sejarah

```

2.3 DaftarKata_2511532010 dan DaftarKataDriver_2511532010

```
1 package pekan2_2511532010;
2
3 import java.util.ArrayList;
4
5 public class DaftarKata_2511532010 {
6     private final ArrayList<String> data;
7
8     // Konstruktor: inisialisasi list kosong
9     public DaftarKata_2511532010() {
10         this.data = new ArrayList<>();
11     }
12
13     /** Menambahkan elemen di akhir list. */
14     public void tambah_2511532010(String elemen) {
15         data.add(elemen);
16     }
17
18     /** Menambahkan elemen pada indeks tertentu (menvisipkan). */
19     public void tambahPada_2511532010(int index, String elemen) {
20         data.add(index, elemen);
21     }
22
23     /**
24      * Mengubah elemen pada posisi 'index' menjadi 'nilaiBaru'.
25      * Bertindak sebagai "setter" untuk elemen tertentu.
26      */
27     public void ubahElemen_2511532010(int index, String nilaiBaru) {
28         data.set(index, nilaiBaru);
29     }
30
31     /**
32      * Menghapus elemen pada posisi 'index' dan mengembalikan nilai yang dihapus.
33      */
34     public String hapusElemen_2511532010(int index) {
35         return data.remove(index);
36     }
37     public void iterasiCetak_2511532010() {
38         for (int i = 0; i < data.size(); i++) {
39             System.out.print(data.get(i) + " ");
40         }
41     }
42
43     /** Mengambil elemen berdasarkan indeks. */
44     public String get(int index) {
45         return data.get(index);
46     }
47
48     // Representasi string agar mudah dicetak
49     @Override
50     public String toString() {
51         return data.toString();
52     }
53 }
54
55 /**
56  * Melakukan iterasi dan mencetak setiap elemen dalam format: [index] nilai
57  * (Metode ini tidak mengembalikan nilai; hanya demonstrasi iterasi).
58  */
```

```

1 package pekan2_2511532010;
2
3 public class DaftarKataDriver_2511532010 {
4
5
6     public static void main(String[] args) {
7         DaftarKata_2511532010 al = new DaftarKata_2511532010();
8
9         // Menambah elemen (akhir)
10        al.tambah_2511532010("Kami");
11        al.tambah_2511532010("Informatika");
12
13        // Menvisipkan elemen pada indeks 1
14        al.tambahPada_2511532010(1, "Mahasiswa");
15
16        // Cetak isi awal
17        System.out.println("Awal          : " + al);
18
19        // Mengubah elemen (index 1)
20        al.ubahElemen_2511532010(1, "Departemen");
21        System.out.println("Setelah Ubah   : " + al);
22
23        // Menghapus elemen (hapus index 0)
24        String terhapus = al.hapusElemen_2511532010(0);
25        System.out.println("Terhapus      : " + terhapus);
26        System.out.println("Setelah Hapus  : " + al);
27
28        // Iterasi pada ArrayList (cetak setiap elemen)
29        System.out.print("Iterasi: ");
30        al.iterasiCetak_2511532010();
31        System.out.println();
32
33    }
34 }
35

```

1. Konsep Wrapper Class pada ArrayList

Dalam kelas DaftarKata_2511532010, kita mendeklarasikan `ArrayList<String>` sebagai variabel privat. Strategi kita di sini adalah menyembunyikan kerumitan ArrayList asli dan hanya menyediakan metode-metode yang memang kita butuhkan. Penggunaan kata kunci `final` pada variabel data memastikan bahwa referensi daftar ini tidak akan tertukar atau diganti dengan list lain setelah dibuat.

2. Konstruktor Inisialisasi

Saat kita memanggil `new DaftarKata_2511532010()`, kita secara otomatis menyiapkan sebuah "wadah" kosong di memori. Kita tidak perlu menentukan ukuran awal karena `ArrayList` di dalamnya akan menangani alokasi ruang secara otomatis sesuai kebutuhan data kita.

3. Metode Penambahan Standar (Append)

Metode `tambah_2511532010` adalah cara paling dasar yang kita gunakan untuk memasukkan kata. Setiap kali metode ini dipanggil, elemen baru akan diletakkan di posisi paling ujung (akhir) dari daftar yang kita miliki.

4. Teknik Penyisipan Elemen (Insertion)

Metode `tambahPada_2511532010` memberikan kita fleksibilitas lebih. Jika kita ingin menyelipkan sebuah kata di tengah-tengah (seperti menyelipkan kata "Mahasiswa" di antara "Kami" dan "Informatika"), kita cukup menentukan indeksnya. Java akan secara otomatis menggeser elemen-elemen di belakangnya ke kanan untuk memberi ruang bagi kata baru tersebut.

5. Logika Perubahan Data (Setter)

Kita menyediakan metode `ubahElemen_2511532010` yang memanfaatkan fungsi `.set()` dari `ArrayList`. Berbeda dengan penyisipan, metode ini akan menimpa (replace) nilai lama pada indeks tertentu dengan nilai baru yang kita inginkan tanpa mengubah ukuran total daftar kita.

6. Penghapusan dan Pengambilan Nilai

Metode `hapusElemen_2511532010` milik kita berfungsi ganda. Selain

menghapus data dari daftar, ia juga mengembalikan (return) nilai yang baru saja dihapus. Ini sangat berguna jika kita ingin menampilkan pesan konfirmasi kepada pengguna mengenai kata apa yang baru saja hilang dari sistem.

7. Iterasi Manual dan Akses Data

Dalam `iterasiCetak_2511532010`, kita menggunakan perulangan `for` klasik dengan metode `.get(i)`. Ini menunjukkan cara kita mengakses elemen satu per satu berdasarkan alamat indeksnya untuk dicetak ke layar tanpa tanda kurung siku bawaan `ArrayList`.

8. Pemanfaatan Delegasi `toString`

Kita tidak perlu repot-repot menyusun format `String` secara manual di sini. Kita cukup melakukan "delegasi" dengan memanggil `data.toString()`. Karena `ArrayList` bawaan Java sudah memiliki format cetak yang bagus (menggunakan kurung siku dan koma), kita bisa langsung memanfaatkannya untuk mempermudah pengecekan isi daftar.

9. Alur Eksekusi pada Driver

Pada kelas `Driver_2511532010`, kita mempraktikkan skenario penggunaan nyata. Kita mulai dengan menambah kata, menyisipkan di tengah, lalu mengubah salah satu elemen. Setiap langkah ini menunjukkan betapa dinamisnya struktur data yang kita gunakan dibandingkan dengan array statis.

10. Efisiensi Pengelolaan Memori

Poin terakhir yang perlu kita perhatikan adalah bagaimana program ini menangani pergeseran data. Baik saat menyisipkan maupun menghapus di indeks 0

(seperti saat menghapus kata "Kami"), kita tidak perlu menulis algoritma geser manual. Semua kerumitan indeks yang berubah sudah diatur sepenuhnya oleh kelas ArrayList yang kita bungkus di dalam kelas DaftarKata ini.

HASIL :

```
Awal      : [Kami, Mahasiswa, Informatika]
Setelah Ubah : [Kami, Departemen, Informatika]
Terhapus   : Kami
Setelah Hapus : [Departemen, Informatika]
Iterasi: Departemen Informatika
```

BAB III

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Hasil praktikum ini menunjukkan bahwa penggunaan ArrayList menawarkan fleksibilitas yang jauh lebih unggul dibandingkan array konvensional, terutama dalam mengelola kumpulan data yang bersifat dinamis. Melalui pemanfaatan metode `add()`, `remove()`, dan `size()`, pengelolaan elemen data menjadi lebih efisien karena praktikan tidak lagi dibatasi oleh kapasitas memori yang statis. Adanya fitur otomatisasi seperti `resizable-array` serta pergeseran indeks otomatis (`automatic shifting`) saat terjadi penghapusan data secara signifikan menyederhanakan kompleksitas logika kode. Hal ini berperan penting dalam mencegah galat fatal seperti `ArrayIndexOutOfBoundsException` yang sering menjadi kendala utama pada struktur array statis.

Selain aspek teknis, praktikum ini memperdalam pemahaman mengenai konsep Object-Oriented Programming (OOP) melalui integrasi ArrayList dengan objek kompleks, seperti pada kasus data mahasiswa dan mobil. Penerapan `Generic` terbukti krusial dalam menjamin keamanan tipe data (`type safety`), sementara penggunaan fitur modern seperti `lambda expression` (`removeIf`) dan metode modular berhasil meningkatkan keterbacaan serta performa program. Secara keseluruhan, penguasaan terhadap Java Collection Framework menjadi landasan fundamental bagi praktikan untuk merancang aplikasi yang lebih adaptif, stabil, dan profesional dalam menangani berbagai skenario input data yang kompleks di masa mendatang.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorials — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Class Music_2511532010

```
public class Musik_2511532010 {  
    private String judul_2010;  
    private String penyanyi_2010;  
    private int durasi_2010;  
  
    // CONSTRUCTOR (Bagian yang harus diperbaiki)  
    public Musik_2511532010(String judul_2010, String penyanyi_2010, int durasi_2010) {  
        this.judul_2010 = judul_2010;  
        this.penyanyi_2010 = penyanyi_2010;  
        this.durasi_2010 = durasi_2010;}  
  
    public String getJudul_2010() { return judul_2010; }  
    public String getPenyanyi_2010() { return penyanyi_2010; }  
    public int getDurasi_2010() { return durasi_2010; }  
  
    public void setJudul_2010(String judul_2010) { this.judul_2010 = judul_2010; }  
    public void setPenyanyi_2010(String penyanyi_2010) { this.penyanyi_2010 = penyanyi_2010; }  
    public void setDurasi_2010(int durasi_2010) { this.durasi_2010 = durasi_2010; }}
```

2. Class Playlist_2511532010

```

import java.util.ArrayList;
import java.util.Scanner;

public class Playlist_2511532010 {
    public static void main(String[] args) {
        ArrayList<Musik_2511532010> daftarLagu = new ArrayList<>();
        Scanner input = new Scanner(System.in);
        int pilih;

        do {
            System.out.println("\n=== Playlist Musik NIM: 2511532010 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Lihat Playlist");
            System.out.println("3. Hapus Lagu");
            System.out.println("4. Cek Kapasitas");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");

            pilih = input.nextInt();
            input.nextLine();

            switch (pilih) {
                case 1:
                    //Tambah lagu
                    System.out.print("Masukkan Judul: ");
                    String judul = input.nextLine();

                    System.out.print("Masukkan Penyanyi: ");
                    String penyanyi = input.nextLine();

                    System.out.print("Masukkan Durasi (detik): ");
                    int durasi = input.nextInt();
                    input.nextLine();

                    Musik_2511532010 laguBaru = new Musik_2511532010(judul, penyanyi, durasi);
                    daftarLagu.add(laguBaru);
                    System.out.println("Data berhasil ditambahkan!");
                    break;

                case 2:
                    // Lihat Playlist
                    System.out.println("\n--- Daftar Lagu ---");
                    if (daftarLagu.isEmpty()) {
                        System.out.println("Playlist masih kosong.");
                    } else {
                        for (int i = 0; i < daftarLagu.size(); i++) {
                            Musik_2511532010 m = daftarLagu.get(i);
                            System.out.println((i + 1) + ". " + m.getJudul_2010() + " - " +
                                m.getPenyanyi_2010() + " (" + m.getDurasi_2010() + " detik");
                        }
                    }
                    break;

                case 3:
                    // Hapus lagu
                    if (daftarLagu.isEmpty()) {
                        System.out.println("Tidak ada lagu untuk dihapus.");
                    } else {
                        System.out.print("Masukkan nomor urut yang ingin dihapus (1-" + daftarLagu.size() + "): ");
                        int nomor = input.nextInt();
                        input.nextLine();

                        if (nomor > 0 && nomor <= daftarLagu.size()) {
                            daftarLagu.remove(nomor - 1);
                            System.out.println("Lagu berhasil dihapus!");
                        } else {
                            System.out.println("Nomor tidak valid!");
                        }
                    }
                    break;

                case 4:
                    // Cek Kapasitas
                    System.out.println("Total lagu dalam playlist saat ini: " + daftarLagu.size());
                    break;

                case 5:
                    //Keluar
                    System.out.println("Terima kasih! Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak tersedia.");
            }
        } while (pilih != 5);

        input.close();
    }
}

```

HASIL :

```

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 1
Masukkan Judul: Bad Guy
Masukkan Penyanyi: Billie Eilish
Masukkan Durasi (detik): 200
Data berhasil ditambahkan!

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 1
Masukkan Judul: Poker Face
Masukkan Penyanyi: Lady Gaga
Masukkan Durasi (detik): 210
Data berhasil ditambahkan!

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 4
Total lagu dalam playlist saat ini: 2

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 2

--- Daftar Lagu ---
1. Bad Guy - Billie Eilish (200 detik)
2. Poker Face - Lady Gaga (210 detik)

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 3
Masukkan nomor urut yang ingin dihapus (1-2): 1
Lagu berhasil dihapus!

=== Playlist Musik NIM: 2511532010 ===
1. Tambah Lagu
2. Lihat Playlist
3. Hapus Lagu
4. Cek Kapasitas
5. Keluar
Pilihan: 2
|
--- Daftar Lagu ---
1. Poker Face - Lady Gaga (210 detik)

```